

Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 43: Mid-way evaluation and Machine Learning Intro:
October 21, 2025

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Recap

- Bias can be added in ways that improve prediction
 - it does this by removing collinearity
 - thereby making the model stable
 - and more generalisable

Learning goals and outline

Mid-way evaluation and Machine Learning Intro

1) Learning some early *classification* methods

- Perceptron and ADALine
- Classification depends on having a quantiser function
- Predicting categories instead of response

2) Learning how linear regression (with biasing penalties) can be constructed and cross-validated

postponed until next time

Mid-way evaluation

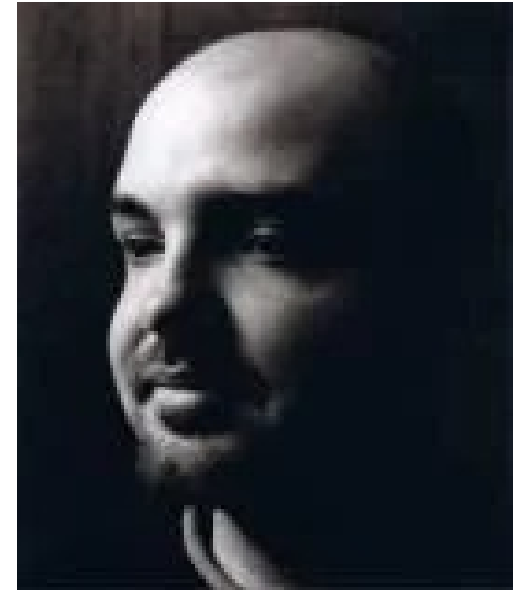
~10 min

- Write one thing you liked about the course so far
- What one thing would you change?
 - Be actionable!

I'll summarise the feedback on the two points, and what we can change, next lecture

But first!

Study Intensity!!



Encoding vs decoding

- ENCODING

- Describes category
- Rich data necessitates strategies for controlling the Family-wise error rate
- Allows for inferences about specific features, e.g. *channel, voxel, frequency, time*

- DECODING

- Predicts category
- Rich data can be reduced to a few data points, reducing the impact of the Family-wise error rate
- Does not (easily) allow for inferences about specific features (black box)

Let's have a look at an iris flower

se·pal 🔊 (sē'pəl)

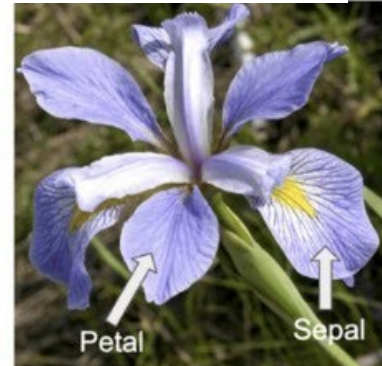
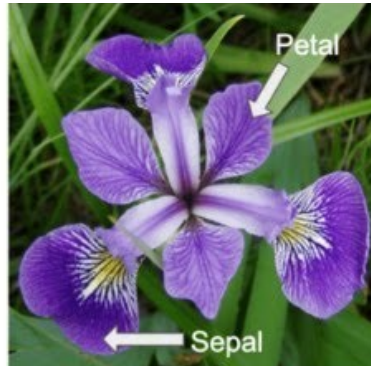
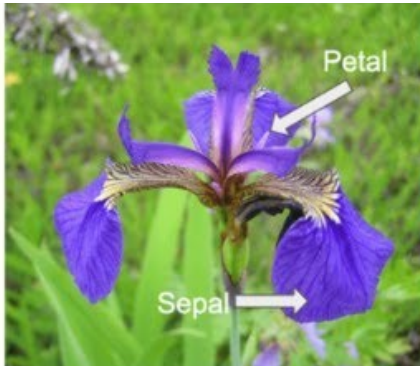
n.

One of the usually green leaflike structures composing the outermost part of a flower. Sepals often enclose and protect the bud and may remain after the fruit forms.

pet·al 🔊 (pět'l)

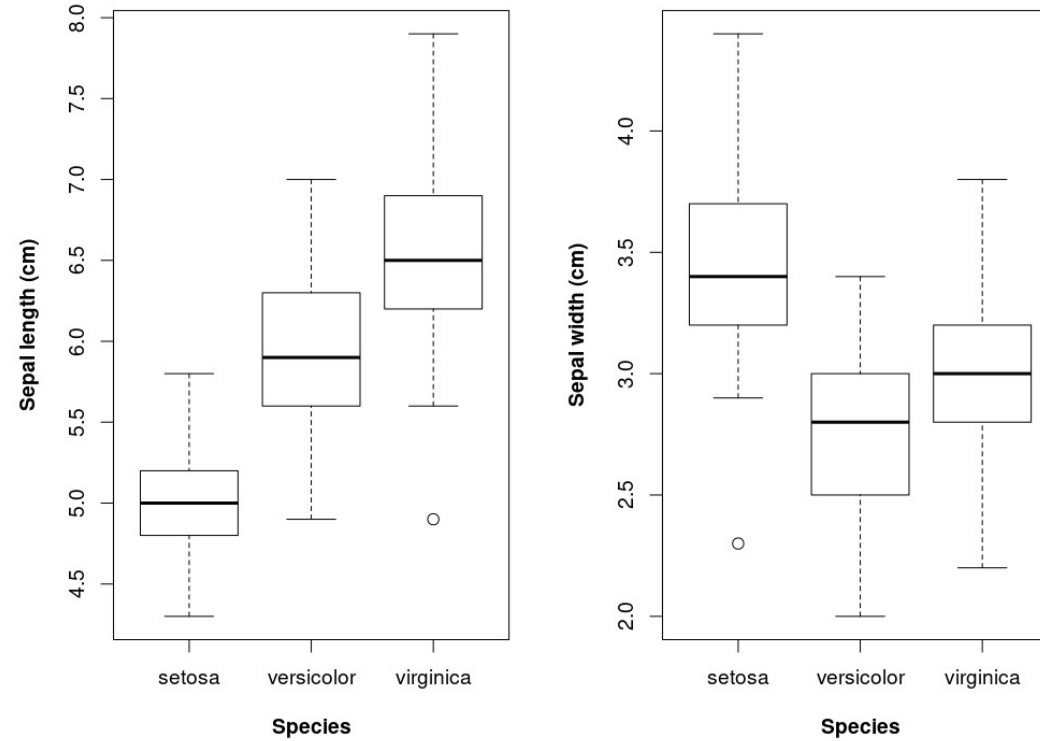
n.

One of the often brightly colored parts of a flower immediately surrounding the reproductive organs; a division of the corolla. —



Encoding

Encoding: *describing the category based on the measurement variable*



Encoding

```
> print(anova(lm(Sepal.Length ~ Species, data=iris)))
```

Analysis of Variance Table

Response: Sepal.Length

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Species	2	63.212	31.606	119.26	< 2.2e-16 ***
Residuals	147	38.956	0.265		

```
> print(anova(lm(Sepal.Width ~ Species, data=iris)))
```

Analysis of Variance Table

Response: Sepal.Width

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Species	2	11.345	5.6725	49.16	< 2.2e-16 ***
Residuals	147	16.962	0.1154		

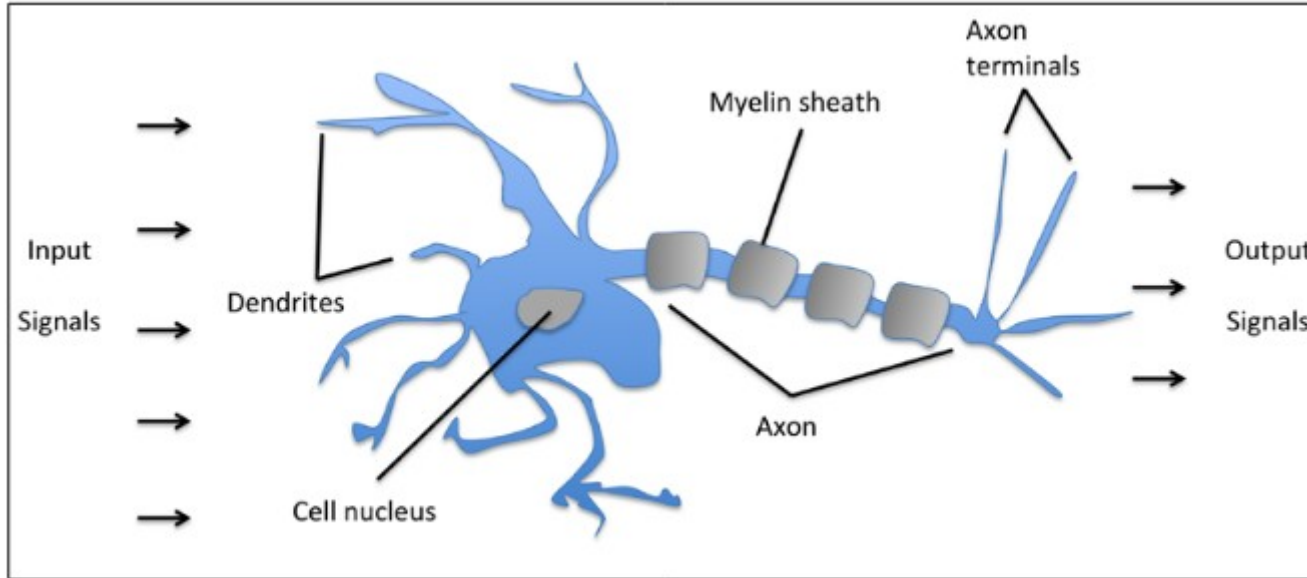
Encoding: statistics can then help you make *inferences* as to whether *measurements* differ between the *categories*, i.e. whether *categories* capture meaningful differences

The Perceptron

DECODING

Raschka S (2015) Python Machine Learning. Packt Publishing Ltd

Black box idea



$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

$$z = w_1 x_1 + \dots + w_m x_m$$

Question: what do x , w and z correspond to in the above picture of the *Perceptron*?

(p. 18: Raschka, 2015)

Prediction/classification rule

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}$$

Perceptron fires

Perceptron doesn't fire

θ is a pre-specified threshold

Prediction/classification rule

$$w_0 = -\theta$$

$$x_0 = 1$$

$$z = w_0 x_0 + w_1 x_1 + \dots + w_m x_m = \mathbf{w}^T \mathbf{x}$$

$$z = -\theta + w_1 x_1 + \dots + w_m x_m = \mathbf{w}^T \mathbf{x}$$

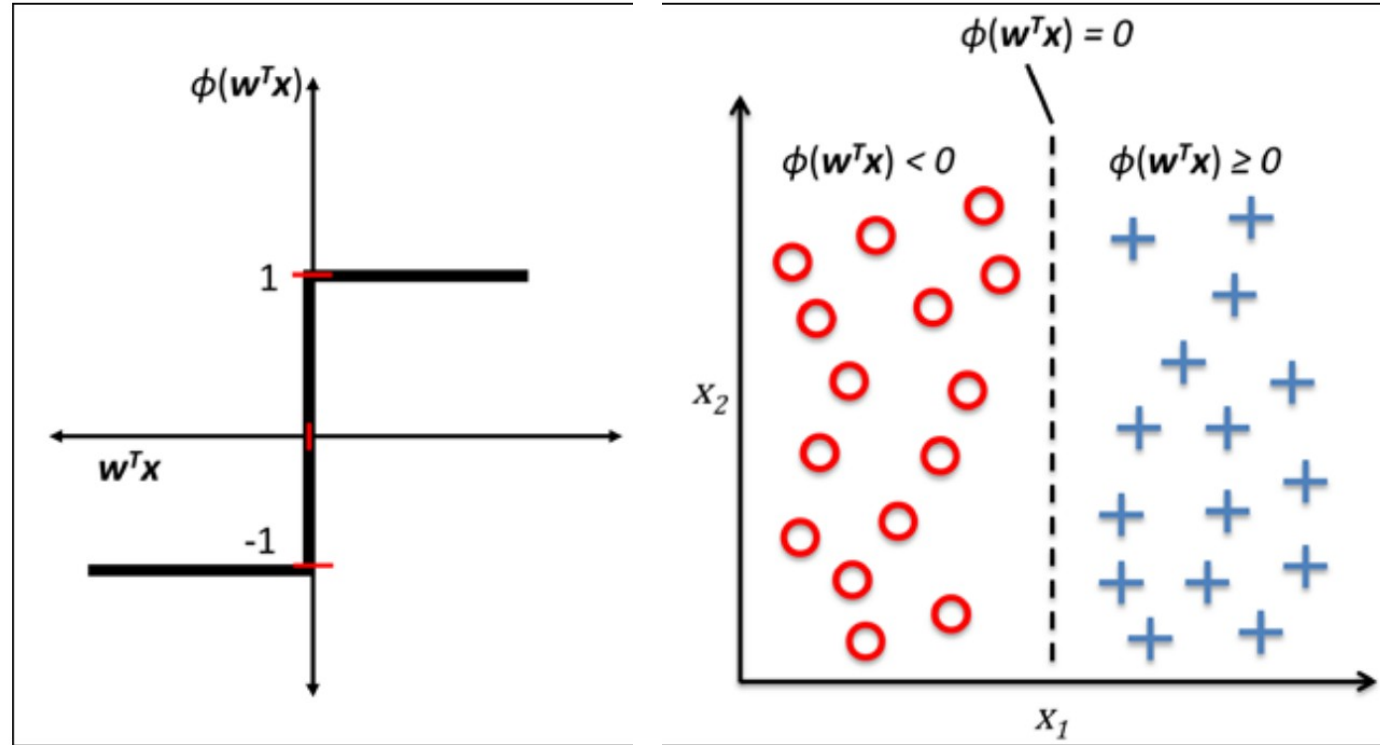
$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Perceptron fires

Perceptron doesn't fire

Perceptron classification

We want to find $\mathbf{w}^T \mathbf{x}$ that achieves this separation



In *Python* (in 2021)

```
class Perceptron(object):  
    """ Perceptron classifier  
  
    Parameters  
    -----  
    eta : float  
        Learning rate (between 0.0 and 1.0)  
    n_iter : int  
        Passes over the training dataset.  
  
    Attributes  
    -----  
    w_ : 1d-array  
        Weights after fitting.  
    errors_ : list  
        Number of misclassifications in every epoch.  
  
    """
```

Special definition that indicates what the object (*Perceptron*) can be initialised with

```
def __init__(self, eta=0.01, n_iter=10):  
    self.eta = eta  
    self.n_iter = n_iter
```

```
ppn = Perceptron(eta=0.1, n_iter=10)
```

Specifying methods of *Perceptron*

1. Initialize the weights to 0 or small random numbers.
2. For each training sample $\mathbf{x}^{(i)}$ perform the following steps:
 1. Compute the output value $\hat{y}$.
 2. Update the weights.

```
def fit(self, X, y):  
    """ Fit training data.  
  
    Parameters  
    -----  
    X : {array-like}, shape = [n_samples, n_features]  
        Training vectors, where n_samples  
        is the number of samples and  
        n_features is the number of features.  
    y : array-like, shape = [n_samples]  
        Target values.  
  
    Returns  
    -----  
    self : object  
  
    """  
    self.w_ = np.zeros(1 + X.shape[1])  
    self.errors_ = []  
  
    for _ in range(self.n_iter):  
        errors = 0  
        for xi, target in zip(X, y):  
            update = self.eta * (target - self.predict(xi))  
            self.w_[1:] += update * xi  
            self.w_[0] += update  
            errors += int(update != 0.0)  
        self.errors_.append(errors)  
    return self
```

Compute the output value $\hat{y}$



```
def net_input(self, X):  
    """Calculate net input"""  
    return np.dot(X, self.w_[1:]) + self.w_[0]  
  
def predict(self, X):  
    """Return class label after unit step"""  
    return np.where(self.net_input(X) >= 0.0, 1, -1)
```



$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}$$

When we are right

$$\Delta w_j = \eta (\overset{\text{real label}}{\boxed{-1}} - \underset{\text{predicted label}}{\boxed{-1}}) x_j^{(i)} = 0$$

$$\Delta w_j = \eta (\overset{\text{real label}}{\boxed{1}} - \underset{\text{predicted label}}{\boxed{1}}) x_j^{(i)} = 0$$

```
update = self.eta * (target - self.predict(xi))  
self.w_[1:] += update * xi  
self.w_[0] += update
```

Δw_j : change in weight

η : learning rate

When we are wrong

$$\Delta w_j = \eta (\overset{\text{real label}}{\boxed{1}} - \underset{\text{predicted label}}{\boxed{-1}}) x_j^{(i)} = \eta (2) x_j^{(i)}$$

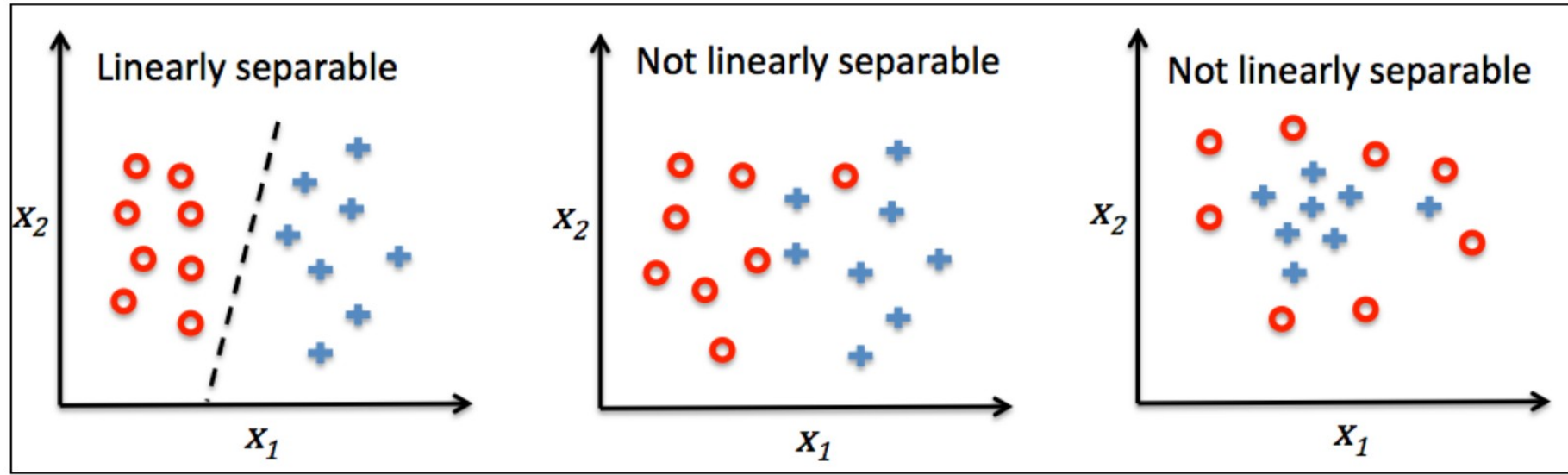
```
update = self.eta * (target - self.predict(xi))
self.w_[1:] += update * xi
self.w_[0] += update
```

$$\Delta w_j = \eta (\overset{\text{real label}}{\boxed{-1}} - \underset{\text{predicted label}}{\boxed{1}}) x_j^{(i)} = \eta (-2) x_j^{(i)}$$

Δw_j : change in weight

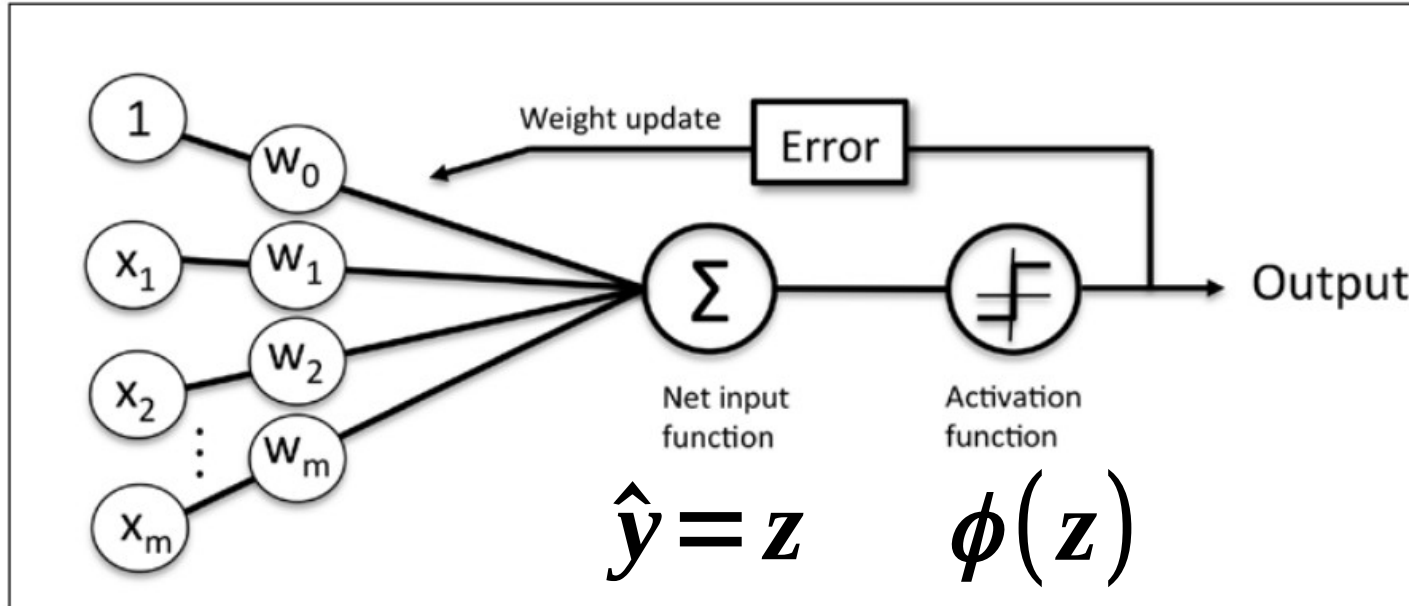
η : learning rate

Convergence only possible when linearly separable



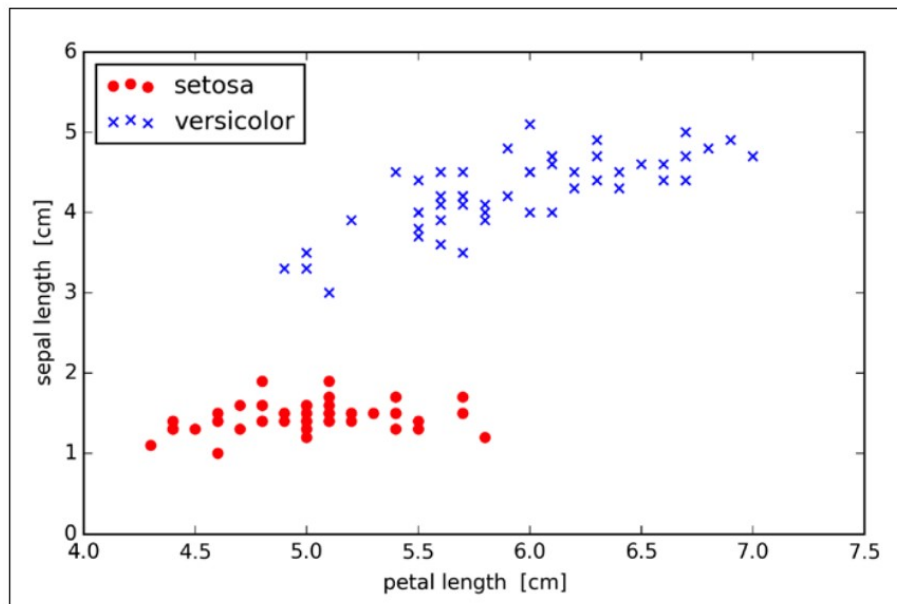
(p. 23: Raschka, 2015)

Perceptron: Graphical summary

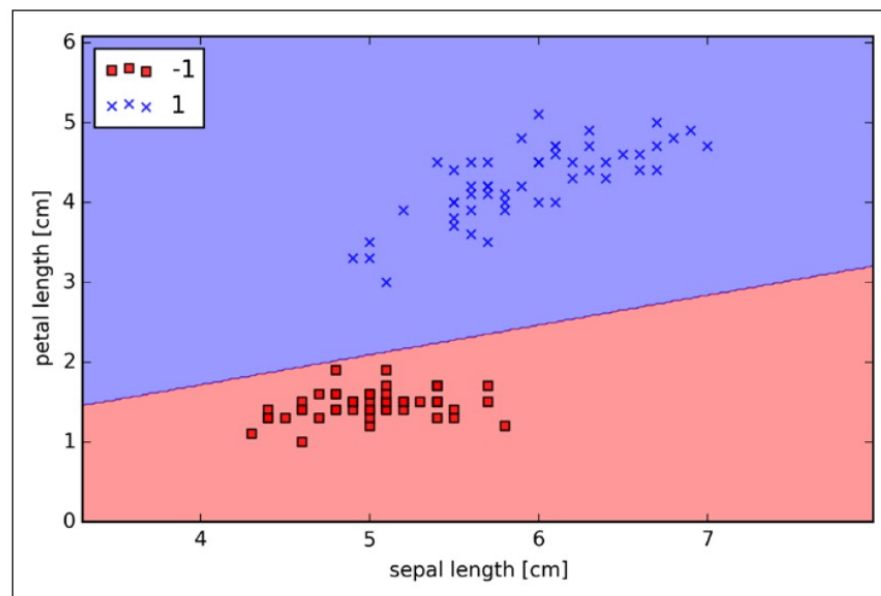


(p. 24: Raschka, 2015)

An example



```
In [153]: ppn.w_  
Out[153]: array([-0.04 , -0.068,  0.182])
```



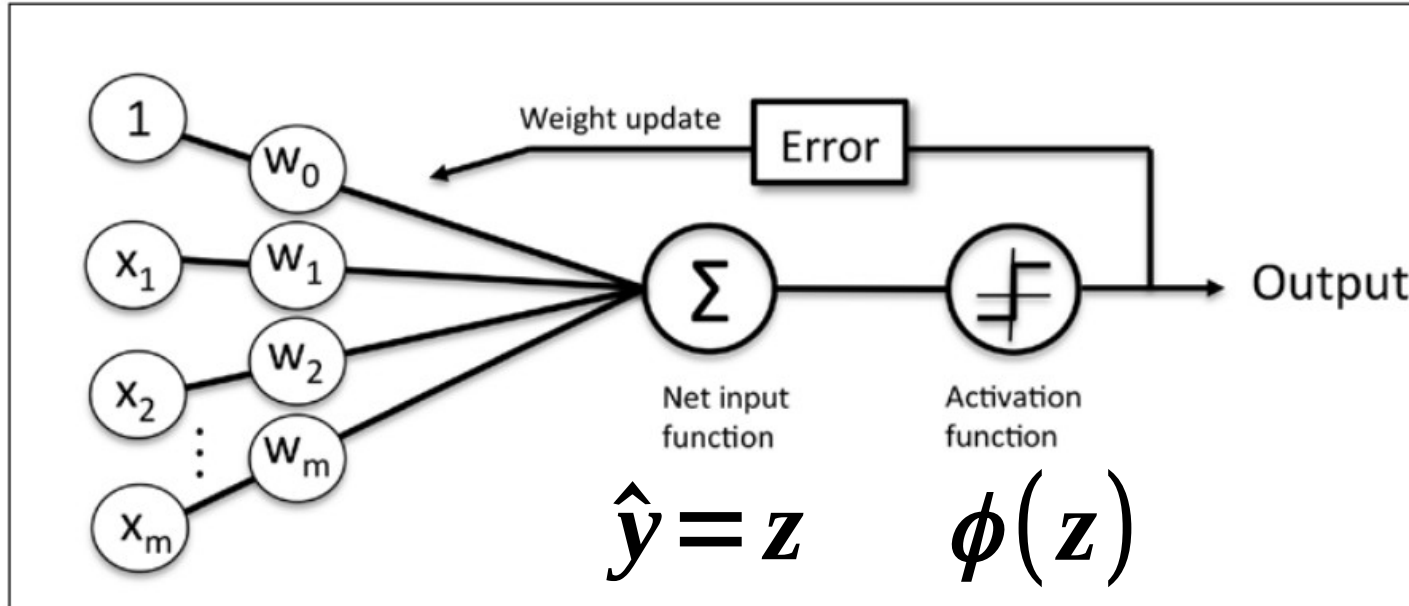
```
In [155]: X_subset[0, :]  
Out[155]: array([5.1, 1.4])
```

$$\hat{y}_1 = -0.04 - 0.068 \cdot 5.1 + 0.182 \cdot 1.4 = -0.132$$

```
In [156]: ppn.net_input(X_subset[0, :])  
Out[156]: -0.13199999999999953
```

(p. 29 & p. 32: Raschka, 2015)

Perceptron: Graphical summary



(p. 24: Raschka, 2015)

Live coding Perceptron

(*W43_live_coding.py*)

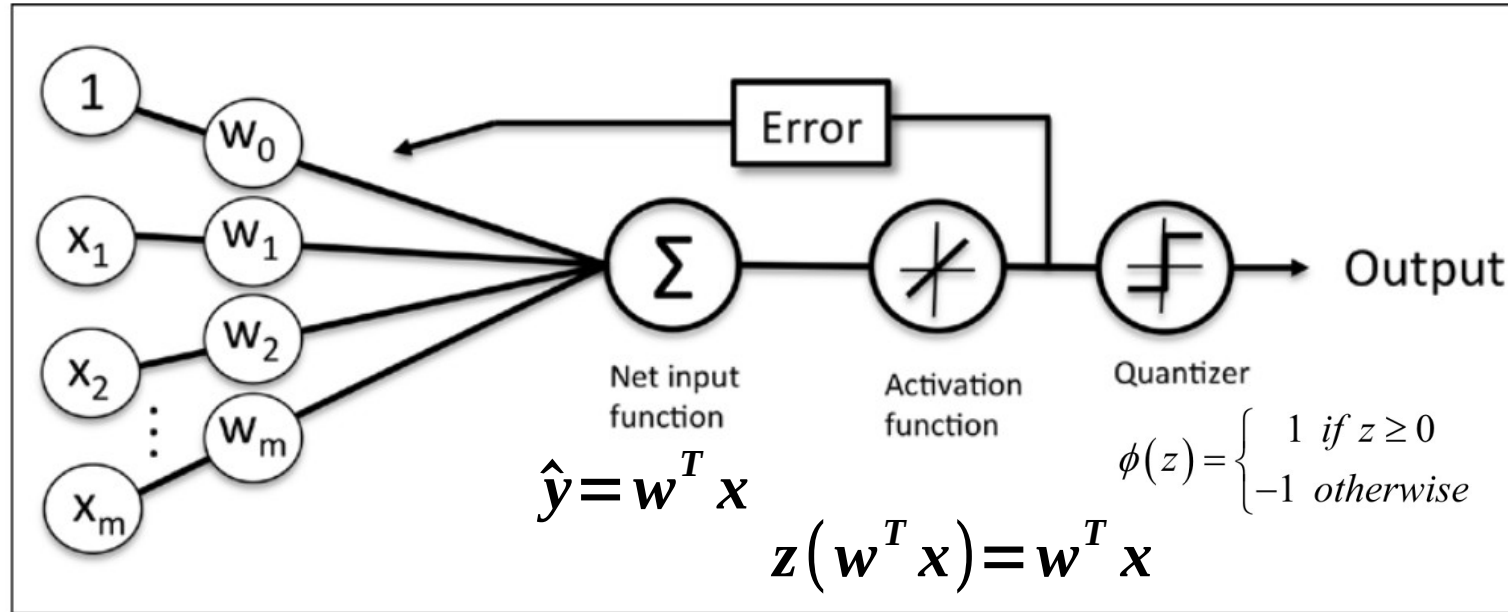
ADALINE

ADaptive Linear NEuron

DECODING

Raschka S (2015) Python Machine Learning. Packt Publishing Ltd

ADaptive Linear NEuron (ADALINE)



$$\mathbf{w}^T \mathbf{x} = w_0 x_0 + w_1 x_1 + \dots + w_{m-1} x_{m-1} + w_m x_m$$

ADALINE Gradient descent

```
def __init__(self, eta=0.01, n_iter=50):  
    self.eta = eta  
    self.n_iter = n_iter
```

```
class AdalineGD(object):  
    """ ADaptive LInear NEuron classifier  
  
    Parameters  
    -----  
    eta : float  
        Learning rate (between 0.0 and 1.0)  
    n_iter : int  
        Passes over the training dataset.  
  
    Attributes  
    -----  
    w_ : 1d-array  
        Weights after fitting.  
    errors_ : list  
        Number of misclassifications in every epoch.  
  
    """
```

Methods

$$\hat{y} = w^T x$$

$$z(w^T x) = w^T x$$

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

```
def net_input(self, X):  
    """Calculate net input"""  
    return np.dot(X, self.w_[1:]) + self.w_[0]  
  
def activation(self, X):  
    """Computer linear activation"""  
    return self.net_input(X)  
  
def predict(self, X):  
    """Return class label after unit step"""  
    return np.where(self.activation(X) >= 0.0, 1, -1)
```


The *fit* method

eta (η): learning rate (a constant)

output: $X\hat{\beta} = \hat{y}$

errors: $y - \hat{y}$

X.T.dot(errors): $X^T \cdot (y - \hat{y})$

$X^T \cdot (y - \hat{y}) = \Delta w_1 + \Delta w_2 + \dots + \Delta w_{m-1} + \Delta w_m$

cost function: $(\sum (y - \hat{y})^2) / 2$

```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    -----
    X : {array-like}, shape = [n_samples, n_features]
        Training vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

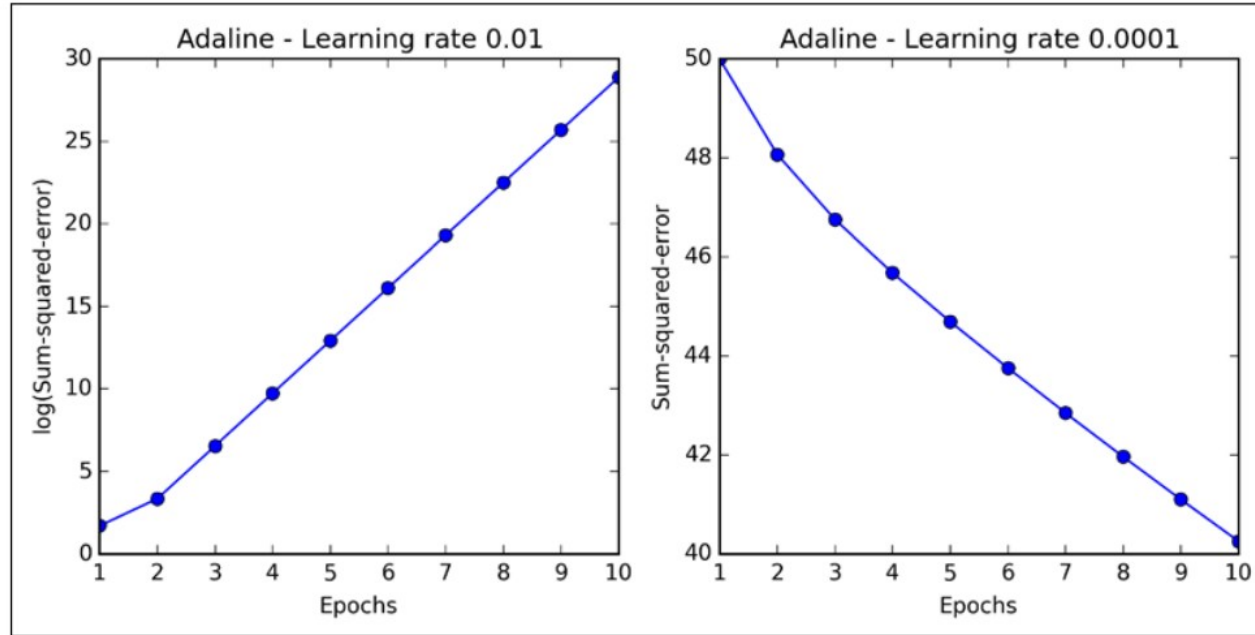
    Returns
    -----
    self : object

    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

Learning rate

Overshooting
the global
minimum

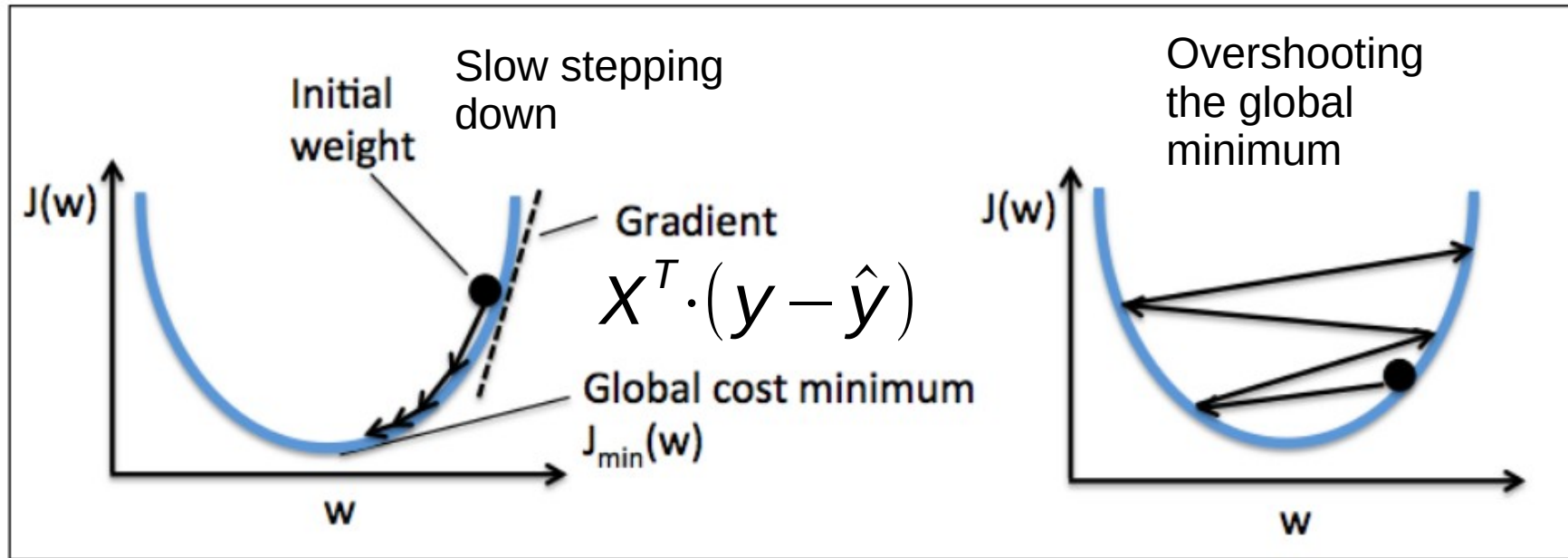


Slow stepping
down

Always check whether it converges

(p. 40: Raschka, 2015)

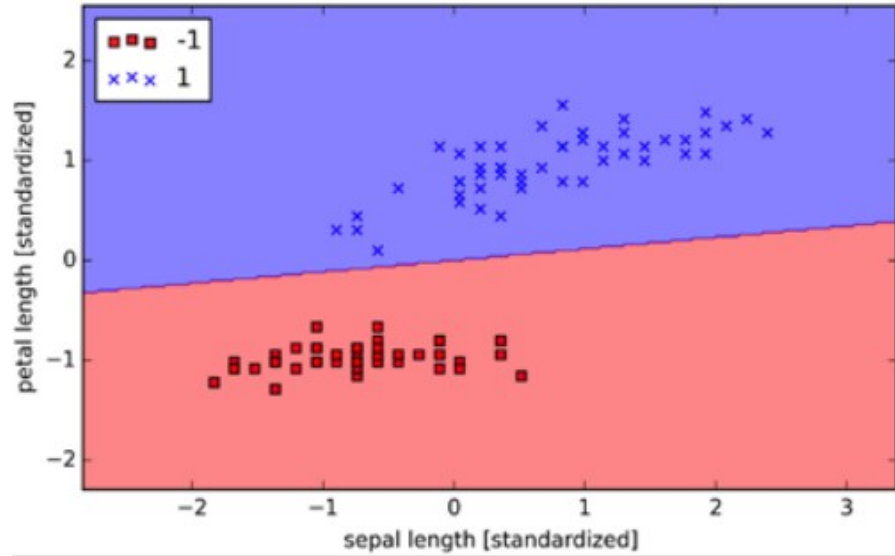
Gradient descent



$$J(w) = (\sum (y - \hat{y})^2) / 2$$

(p. 40: Raschka, 2015)

Adaline - Gradient Descent



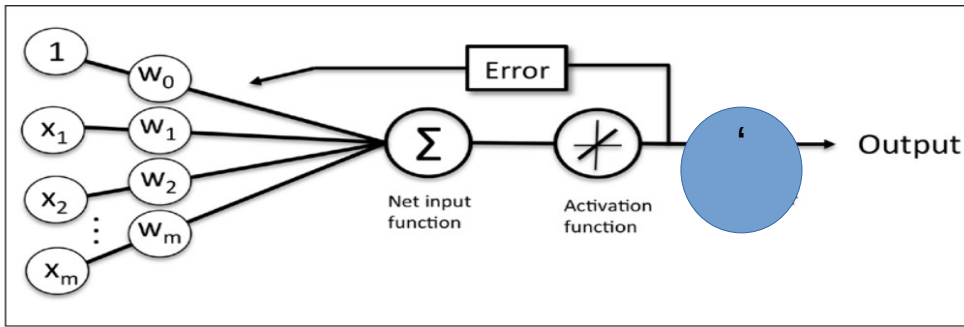
(p. 42: Raschka, 2015)

```
In [163]: ada.w_  
Out[163]: array([-0.40808285, -0.33924452,  0.79202224])
```

```
In [155]: X_subset[0, :]  
Out[155]: array([5.1, 1.4])
```

$$\hat{y}_1 \approx -0.408 - 0.339 \cdot 5.1 + 0.792 \cdot 1.4 = -1.03$$

```
In [164]: ada.net_input(X_subset[0, :])  
Out[164]: -1.029398778794785
```



How does this relate to linear regression?

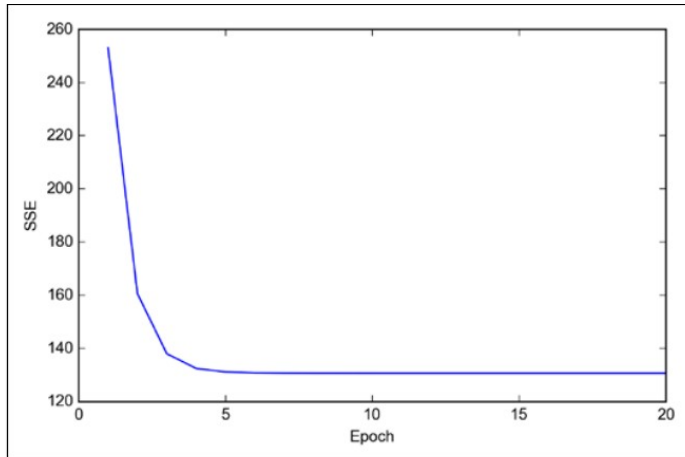
From ADALINE

$$\mathbf{w}^T \mathbf{x} = w_0 x_0 + w_1 x_1 + \dots + w_{m-1} x_{m-1} + w_m x_m$$

Very similar to ADALINE
and when converged will be virtually
identical to the ordinary least
squares solution

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Convergence



```
class LinearRegressionGD(object):  
  
    def __init__(self, eta=0.001, n_iter=20):  
        self.eta = eta  
        self.n_iter = n_iter  
  
    def fit(self, X, y):  
        self.w_ = np.zeros(1 + X.shape[1])  
        self.cost_ = []  
  
        for i in range(self.n_iter):  
            output = self.net_input(X)  
            errors = (y - output)  
            self.w_[1:] += self.eta * X.T.dot(errors)  
            self.w_[0] += self.eta * errors.sum()  
            cost = (errors**2).sum() / 2.0  
            self.cost_.append(cost)  
        return self  
  
    def net_input(self, X):  
        return np.dot(X, self.w_[1:]) + self.w_[0]  
  
    def predict(self, X):  
        return self.net_input(X)
```

ADALINE

```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    -----
    X : {array-like}, shape = [n_samples, n_features]
        Traing vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -----
    self : object

    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

```
def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def activation(self, X):
    """Computer linear activation"""
    return self.net_input(X)

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

```
class LinearRegressionGD(object):
```

```
    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        self.w_ = np.zeros(1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            output = self.net_input(X)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def predict(self, X):
        return self.net_input(X)
```

Live coding ADALINE

(*W43_live_coding.py*)

Interim summary

- We can build simple classification tools using scikit-learn
 - These can give us decision boundaries
 - That we can apply to new data (we haven't done that yet)
- We need to define cost functions
 - These can be conceptually separated from prediction functions (remember ADALINE)

Summary

- We can build simple classification tools using scikit-learn
 - These can give us decision boundaries
 - That we can apply to new data (we haven't done that yet)
- We need to define cost functions
 - These can be conceptually separated from prediction functions (remember ADALINE)
- We can use linear regression to do continuous predictions

Learning goals and outline

Mid-way evaluation and Machine Learning Intro

- 1) Learning some early *classification* methods
 - Perceptron and ADALine
 - Classification depends on having a quantiser function
- 2) Learning how linear *regression* (with biasing penalties) can be constructed and cross-validated

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Next time

- Logistic regression
 - adapted for machine learning
 - aim: predicting categories instead of minimising log-likelihood when generalised to new datasets
- Linear regression
 - adapted for machine learning
 - aim: minimising error when generalised to new datasets

Reading questions

- Chapter 10
 - What is lasso and ridge regression?
 - What is standardisation?
- Chapter 3
 - What is the cost function of logistic regression?
 - What is overfitting and underfitting?